

Midterm Project (Week 5)

QA Implementation & Empirical Analysis

Objective

This midterm evaluates your ability to **translate QA theory into measurable engineering outcomes**.

Task 1: Refine Risk-Based Testing Strategy (20 points)

Task 1.1 Re-evaluate High-Risk Components

Revisit your top 3–5 high-risk modules from Assignment 1.

For EACH module, provide:

Module	Original Risk Score	Observed Issues (A2)	Updated Risk Score	Justification
--------	---------------------	----------------------	--------------------	---------------

You must **justify every change in risk prioritization using empirical data** collected from your automation pipeline (Assignment 2).

Task 1.2 Extract Evidence from Automation Runs

From your CI/CD pipeline and test reports, collect the following:

A. Failed Test Cases

For each failure provide:

- Test name / ID
- Module affected
- Failure type (logic error, validation error, timeout, etc.)
- Frequency (how many runs failed)

B. Flaky Tests (Instability Analysis)

Identify tests that:

- Pass and fail inconsistently across runs

For each flaky test:

- Number of passes vs failures
- Suspected cause (timing issue, dependency, race condition)

C. Coverage Gaps

Use coverage tools to identify:

- Untested functions / endpoints / components
- % coverage per high-risk module

Required:

- Highlight modules with **coverage < 70%**

D. Unexpected System Behavior

Document any:

- Errors NOT predicted in Assignment 1
- New failure patterns
- Performance anomalies during test execution

Task 1.3: Map Evidence to Risk Dimensions

For each module, explicitly evaluate:

- **Likelihood** (↑ if many failures / flaky tests)
- **Impact** (↑ if critical functionality affected)
- **Detectability** (↓ if low coverage or hidden bugs)

Task 2: Expand Automation & Coverage (30 points)

Task 2.1 Extend Test Suite

You must **extend your existing automation suite with 5–10 NEW test cases** that specifically target **previously identified high-risk modules**.

A. Test Case Design Requirements

Each new test case **MUST** include:

- Test ID (e.g., TC-AUTH-EDGE-01)
- Target module (must be high-risk)
- Scenario type (failure / edge / concurrency / invalid input)
- Input data
- Expected output

- Actual result (after execution)

B. Mandatory Test Categories (at least 1–2 per category)

1. Failure Scenarios

Test how the system behaves when things go wrong:

- Invalid credentials (authentication)
- Expired tokens
- API returning error codes (400, 401, 500)

2. Edge Cases

Test boundary conditions:

- Empty input fields
- Extremely large payloads
- Special characters / injection-like inputs

3. Concurrency / Race Conditions

Simulate:

- Multiple simultaneous requests
- Parallel API calls
- Conflicting operations (e.g., double submission)

4. Invalid User Behavior

Examples:

- Skipping required steps
- Repeating actions rapidly
- Accessing restricted endpoints

Task 2.2 Required Test Types (Implementation Depth)

You MUST implement **all three levels of testing**:

A. Unit Tests (Logic-Level)

B. Integration Tests (Module Interaction)

C. End-to-End (E2E) Tests (User Flow)

D. Required Evidence

You must show:

- Code snippets OR repository links
- Test execution logs
- Clear mapping: test → system behavior

Task 2.3 CI/CD Execution

Your automation **MUST** run inside a **CI/CD pipeline** (not manually only).

A. Pipeline Configuration

Use one of:

- GitHub Actions
- GitLab CI/CD
- Jenkins

B. Required Pipeline Behavior

On **every push/commit**, pipeline must:

1. Install dependencies
2. Run ALL tests
3. Generate:
 - Pass/fail results
 - Coverage report
4. Apply quality gates (see 2.4)

C. Required Evidence

You must include:

- Screenshot of pipeline run
- Link to repository
- Logs showing:
 - Test execution
 - Failures (if any)

D. Example Pipeline Flow

Push → Build → Run Tests → Generate Coverage → Apply Quality Gates
→ Pass/Fail

Task 2.4 Quality Gates (Definition + Evaluation)

You must define **explicit, measurable thresholds** that determine whether your system meets quality standards.

A. Mandatory Quality Gates

Define at least **3 gates**, such as:

1. **Test Pass Rate**
 - Example: $\geq 90\%$
2. **Code Coverage**
 - Example: $\geq 70\%$ overall OR $\geq 60\%$ in high-risk modules
3. **Critical Failures**
 - Example: 0 critical test failures allowed

C. Critical Analysis

You must answer:

- Were your thresholds **too strict or too lenient**?
- Did the system fail due to:
 - Poor code quality?
 - Insufficient tests?
 - Unrealistic thresholds?

Task 3: Metrics Collection (20 points)

You **MUST** collect and present **real numerical data**.

3.1 Required Metrics

Coverage

- % of high-risk modules covered
- Tool examples: coverage.py, JaCoCo, Istanbul

Defect Detection

- Number of defects found
- Map defects → risk level

Efficiency

- Test execution time (before vs after improvements)
- Pipeline runtime

Stability

- Flaky test rate (if applicable)

Task 4: Comparative Analysis (15 points)

4.1 Planned vs Actual

Compare:

Aspect	Planned (A1)	Actual (A2/A3)	Gap

4.2 Required Insights

You MUST identify:

- Incorrect assumptions in planning
- Missing test scenarios
- Inefficient automation design

Task 5: Midterm Report (15 points)

Structure (MANDATORY)

1. System Description (0.5–1 page)

- Architecture (e.g., microservices, monolith)
- Technologies used
- Key functionalities

2. Methodology (1–1.5 pages)

- Risk-based testing approach
- Test design strategy
- Automation tools (e.g., Selenium, PyTest, Postman)

3. Automation Implementation (1 page)

- CI/CD setup
- Test structure
- Quality gates definition

4. Results (1–1.5 pages)

- Metrics tables
- Graphs (coverage, defects, execution time)
- Screenshots of pipelines/logs

5. Discussion (1 page)

- What worked / what didn't
- Unexpected findings
- Improvements for next phase